به نام خدا

گزارش تمرین کامپیوتری
شماره دو
سیگنال ها و سیستم ها

محمد صدرا عباسی - ۸۱۰۱۰۱۴۶۹

فهرست مطالب

قسمت اول

۱,۲) انتخاب فایل و تنظیم اندازه

۳) تبدیل به تصویر grayscale

توضیح تابع mygrayfun:

۴) تبدیل تصویر grayscale به binary

توضیح تابع mybinaryfun:

۵) حذف نویز و فیلترینگ

توضیح تابع myremoveobject:

۶) قطعه‌بندی و جداسازی کاراکترها

توضیح تابع mysegmentation:

۷) decision making

۸) نمایش و ذخیره نتیجه نهایی

اسکریپت کامل p1.m

خروجی کامل بخش اول

Image 1

Image 2

Image 3

قسمت دوم

پیش پردازش تصویر و حذف نویز های کوچک و بک گراند

بارگذاری دیتابیس

تشخیص کاراکترها

خروجی کامل بخش دوم

قسمت سوم

روش استخراج کادر پلاک از تصویر

پیاده سازی مرحله به مرحله استخراج پلاک

۱) استخراج لبه های عمودی

۲) عملیات مورفولوژیک

۳) فیلترینگ هندسی

اسکریپت کامل p3.m
خروجی کامل بخش سوم
قسمت چهارم
تثبیت فریم‌های ویدیو
توضیح تابع stabilize_frame:
تشخیص حرکت و استخراج خودروها
توضیح تابع clean_foreground:
محاسبه سرعت خودروها
توضیح تابع calculate_speed:
استخراج کادر پلاک با پنجره لغزان و خواندن کاراکترها
توضیح عملکرد و منطق الگوریتم در تابع extract_plate
نمایش و ذخیره نتیجه نهایی ویدیو
اسکریپت کامل p4.m
خروجی بخش چهارم
قسمت پنجم
توضیح استراتژی حل مسئله
توضیح تابع ICrecognition
اسکریپت p5.m
خروجی بخش پنجم

قسمت اول

۱,۲) انتخاب فایل و تنظیم اندازه

با استفاده از تابع uigetfile یک رابط گرافیکی باز می‌شود که آدرس مسیر و نام فایل تصویر مورد نظر را از کاربر دریافت می‌کند.

```
[file, path] = uigetfile({'*.jpg;*.bmp;*.png;*.tif'}, 'Choose an image');
s = [path, file];
picture = imread(s);
picture = imresize(picture, [300, 500]);
```

نام فایل مورد نظر به همراه مسیر کامل آن در متغیر s ترکیب و ذخیره شده و سپس با استفاده از تابع imread به عنوان یک فایل تصویر، که در قالب یک ماتریس سه بعدی شامل مقادیر شدت روشنایی کانال‌های رنگی قرمز، سبز و آبی (R,G,B) است، در Workspace خوانده شده و در متغیر picture ذخیره می‌شود. در نهایت، طول و عرض این تصویر با استفاده از تابع imresize روی مقادیر ۳۰۰ در ۵۰۰ پیکسل تنظیم می‌شود.

۳) تبدیل به تصویر grayscale

در این بخش به منظور کاهش پیچیدگی محاسبات و سادگی بیشتر فرآیند پردازش، تصویر رنگی پلاک را به حالت Grayscale تبدیل می‌کنیم. با این کار، حجم اطلاعات و ابعاد ماتریس اولیه به یک‌سوم کاهش می‌یابد، زیرا به جای سه ماتریس رنگی، صرفاً یک ماتریس یکپارچه را نگه می‌داریم و مقادیر شدت روشنایی آن را به صورت اعداد صحیح بدون علامت ۸ بیتی در یک ماتریس دو بعدی ذخیره می‌کنیم. برای این ترکیب اصولی، از فرمول استاندارد زیر استفاده شده است:

$$Gray_{channel} = 0.299 \times Red_{channel} + 0.578 \times Green_{channel} + 0.114 \times Blue_{channel}$$

توضیح تابع mygrayfun:

```
function grayImage = mygrayfun(image)
  redChannel = image(:,:,1);
  blueChannel = image(:,:,2);
  greenChannel = image(:,:,3);
    grayChannel = (0.299* double(redChannel)) + (0.578* double(greenChannel)) +
(0.114*double(blueChannel));
```

```
  grayImage = uint8(grayChannel);
end
```

در این تابع، ابتدا کانال‌های رنگی قرمز (R)، سبز (G) و آبی (B) به صورت زیرماتریس‌های دوبعدی مجزا از ماتریس سه‌بعدی تصویر ورودی استخراج می‌شوند. سپس برای انجام دقیق محاسبات اعشاری، مقادیر این کانال‌ها ابتدا به نوع double تبدیل شده و با استفاده از ضرایب فرمول ذکر شده با یکدیگر جمع می‌شوند تا ماتریس کانال خاکستری به دست آید. در نهایت، مقادیر محاسبه شده مجدداً به فرمت استاندارد ۸ بیتی بدون علامت تبدیل شده و در ماتریس grayImage ذخیره می‌گردند تا به عنوان خروجی این تابع برگردانده شوند.

خروجی این بخش برای هر سه تصویر نمونه:

```
grayImage = mygrayfun(picture);
imshow(grayImage);
```







۴) تبدیل تصویر grayscale به binary

در این بخش با داشتن تصویر grayscale که تک‌کاناله است، می‌توانیم مقادیر این کانال رنگی را به ازای هر پیکسل با یک مقدار آستانه‌ای (Threshold) مقایسه کرده و اگر مقدار شدت نور پیکسل از این آستانه کمتر بود (بخش‌های تیره مانند متن پلاک)، آن را یک (معادل سفید) و اگر بیشتر یا مساوی بود (بخش‌های روشن مانند پس‌زمینه)، آن را صفر (معادل مشکی) در نظر بگیریم. با این کار تمام جزئیات غیر ضروری، سایه‌ها و رنگ‌های اضافی تصویر را Mask کرده و در نتیجه پردازش برای جداسازی حروف بسیار ساده‌تر می‌شود.
(علت انتخاب رنگ سفید برای بخش هایی مانند نوشته روی پلاک این است که template هایی که در ادامه برای پردازش استفاده می کنیم نیز به همین شکل ذخیره شده اند)

توضیح تابع mybinaryfun:

```
function binaryImage = mybinaryfun(threshold , image)
  binaryImage = image < threshold;
end
```

یک تابع ساده است که منطق مقایسه‌ای گفته شده در بخش قبل را روی تمام پیکسل‌های تصویر پیاده‌سازی کرده و تصویر باینری حاصل را به عنوان خروجی برمی‌گرداند.

تعیین مقدار 100 برای threshold:

این مقدار با روش آزمون و خطا به گونه‌ای تعیین شده است که با در نظر گرفتن شرایط نوری و کیفیت تصاویر ورودی، بهترین و واضح‌ترین خروجی را برای جداسازی حروف از پس‌زمینه در نهایت تولید کند

خروجی این بخش برای هر سه تصویر نمونه:

```
threshold = 100;
binaryImage = mybinaryfun(threshold, grayImage);
imshow(binaryImage);
```







۵) حذف نویز و فیلترینگ

در این بخش باید تصویر باینری به دست آمده از مرحله قبلی را فیلتر کنیم تا عناصر نویزی موجود روی تصویر باعث پردازش‌های اضافه نشوند. برای این کار باید قسمت‌هایی که از پیوستن تعداد پیکسل‌های کمتری تشکیل شده‌اند را به عنوان نویز در نظر گرفته و از تصویر جدا کنیم.

توضیح تابع myremoveobject:

```
function cleanImage = myremoveobject(n, image)
  connectedComp = bwconncomp(image);
  cleanImage = image;
  for i = 1: connectedComp.NumObjects
     if length(connectedComp.PixelIdxList{i}) < n
        cleanImage(connectedComp.PixelIdxList{i}) = 0;
     end
  end
end
```

دستور bwconncomp تمام اجزای به هم پیوسته در تصویر را استخراج می‌کند. سپس به ازای هر شیء متمایزی که این تابع استخراج کرده است، تعداد پیکسل‌های آن با مقداری که به عنوان ورودی (n) گرفته شده مقایسه می‌شود؛ اگر این تعداد کمتر از مقدار ورودی بود، تمام پیکسل‌های آن بخش مقدار صفر می‌گیرند، یعنی به رنگ پس‌زمینه (مشکی) در می‌آیند.

تعیین مقدار ۳۸۰ برای n:

با تست کردن تصاویر نمونه، مشخص شد که این مقدار بهترین فیلترینگ را برای هر سه تصویر انجام می‌دهد. اگر این مقدار افزایش یابد، عناصر مهم شامل اعداد و حروفِ پلاک به اشتباه حذف می‌شوند و اگر کمتر باشد، باقی ماندن برخی عناصر اضافی باعث ایجاد خروجی اشتباه در مراحل بعدی خواهد شد.

خروجی این بخش برای تصاویر نمونه:







۶) قطعه‌بندی و جداسازی کاراکترها

در این بخش باید تک‌تک حروف و اعداد موجود در تصویر فیلتر شده را از یکدیگر جدا کنیم تا بتوانیم آن‌ها را به صورت مجزا برای تشخیص به الگوریتم Template Matching بفرستیم. برای این کار باید هر شیء پیوسته (پیکسل‌های سفید به هم چسبیده) در تصویر را شناسایی کرده و به آن یک Label اختصاص دهیم.

توضیح تابع mysegmentation:

```
function [L, Ne] = mysegmentation(image)
  cc = bwconncomp(image);
  L = zeros(size(image));
  Ne = cc.NumObjects;
  for i = 1:Ne
```

```
    L(cc.PixelIdxList{i}) = i;
  end
end
```

دستور bwconncomp تمام اجزای به هم پیوسته در تصویر را استخراج می‌کند. سپس یک ماتریسِ تماماً صفر، هم‌اندازه با تصویر ورودی به نام L ایجاد می‌شود. در ادامه، تعداد کل اشیای یافت شده در متغیر Ne ذخیره می‌گردد. الگوریتم با استفاده از یک حلقه روی تمام اشیای استخراج شده حرکت کرده و به ازای هر شیء، تمام پیکسل‌های مربوط به آن را در ماتریس L با یک عدد صحیح و منحصربه‌فرد (از ۱ تا Ne) مقداردهی می‌کند.

۷) decision making

در این بخش، هدف این است که با استفاده از دیتابیس حروف و اعداد انگلیسی که در اختیار داریم، تشخیص دهیم هر Segment از تصویر پلاک دقیقاً چه کاراکتری است. برای این کار از روش Template Matching و محاسبه Correlation بین هر قطعه از تصویر و تمام المان‌های موجود در دیتابیس استفاده می‌کنیم.

توضیح کد و منطق پیاده‌سازی:

```
final_output = [];
for n = 1:Ne
  [r, c] = find(L == n);
  Y = cleanImage(min(r):max(r), min(c):max(c));
  %imshow(Y);
  Y = imresize(Y, [42, 24]);
   totalLetters = size(TRAIN, 2);
  ro = zeros(1, totalLetters);
  for k = 1:totalLetters
    ro(k) = corr2(double(TRAIN{1, k}), double(Y));
  end
   [MAXRO, pos] = max(ro);
  if MAXRO > 0.5
    out = TRAIN{2, pos};
```

```
    final_output = [final_output, out];
  end
end
```

ابتدا تمامی تصاویر موجود در Map Set خوانده شده و به همراه نام کاراکترِ متناظرشان در یک آرایه سلولی به نام TRAIN ذخیره می‌شوند. سپس الگوریتم وارد یک حلقه for به طول Ne (تعداد کل کاراکترهای استخراج شده) می‌شود. در هر تکرار از این حلقه:

۱. با استفاده از دستور find(L == n) مختصات پیکسل‌های مربوط به کاراکتر n-ام پیدا شده و آن کاراکتر از تصویر برش می‌خورد و در ماتریس Y قرار می‌گیرد.

۲. تصویر برش خورده Y با استفاده از دستور imresize دقیقاً به ابعاد قالب‌های استاندارد دیتابیس (۴۲ در ۲۴ پیکسل) تغییر اندازه داده می‌شود تا قابل مقایسه باشند.

۳. با استفاده از یک حلقه داخلی و دستور corr2، ضریب همبستگی دو بعدی بین قطعه Y و تک‌تک المان‌های TRAIN محاسبه شده و در بردار ro ذخیره می‌گردد.

۴. دستور max(ro) بالاترین ضریب همبستگی (MAXRO) و شماره اندیس آن (pos) را پیدا می‌کند که نشان‌دهنده شبیه‌ترین کاراکتر دیتابیس به قطعه فعلی است.

تعیین آستانه همبستگی (MAXRO > 0.5):

برای جلوگیری از ثبت نویزهای حاشیه‌ای که ممکن است از مراحل قبل باقی مانده باشند، یک شرط منطقی قرار داده شده است. تنها در صورتی کاراکترِ پیدا شده به final_output اضافه می‌شود که مقدار تشابه آن با یکی از قالب‌های دیتابیس بیشتر از 0.5 (۵۰ درصد) باشد. همچنین این عدد با تست کردن تصاویر نمونه بدست آمده است

۸) نمایش و ذخیره نتیجه نهایی

توضیح کد ذخیره‌سازی:

```
disp(['Detected License Plate: ', final_output]);
fileID = fopen('plate_result.txt', 'w');
fprintf(fileID, '%s\n', final_output);
fclose(fileID);
```

حروف و اعداد استخراج شده در Command کنسول چاپ می‌شوند سپس برای ذخیره در فایل txt، دستور fopen با پارامتر w یک فایل متنی جدید به نام plate_result.txt ایجاد کرده. تابع fprintf final_output را در این فایل می‌نویسد.

اسکریپت کامل p1.m

```
%% 1. Image Selection and Preprocessing
[file, path] = uigetfile({'*.jpg;*.bmp;*.png;*.tif'}, 'Choose an image');
s = [path, file];
picture = imread(s);
picture = imresize(picture, [300, 500]);
```

```
%% 2. Color to Grayscale and Binarization Conversion
grayImage = mygrayfun(picture);
%imshow(grayImage);
threshold = 100;
binaryImage = mybinaryfun(threshold, grayImage);
imshow(binaryImage);
```

```
%% 3. Noise Removal and Filtering Small Objects
cleanImage = myremoveobject(380, binaryImage);
imshow(cleanImage);
```

```
%% 4. Image Segmentation to Separate Characters
[L, Ne] = mysegmentation(cleanImage);
```

```
%% 5. Load Template Map Set for Recognition
files = dir('Map Set');
files([1, 2]) = [];
len = length(files);
TRAIN = cell(2, len);
for i = 1:len
   TRAIN{1, i} = imread(fullfile('Map Set', files(i).name));
   TRAIN{2, i} = files(i).name(1);
end
```

```
%% 6. Decision Making, Template Matching, and Plate Extraction
final_output = [];
for n = 1:Ne
    [r, c] = find(L == n);
    Y = cleanImage(min(r):max(r), min(c):max(c));
    %imshow(Y);
    Y = imresize(Y, [42, 24]);

    totalLetters = size(TRAIN, 2);
    ro = zeros(1, totalLetters);
    for k = 1:totalLetters
        ro(k) = corr2(double(TRAIN{1, k}), double(Y));
    end

    [MAXRO, pos] = max(ro);
    if MAXRO > 0.5
        out = TRAIN{2, pos};
        final_output = [final_output, out];
    end
end
disp(['Detected License Plate: ', final_output]);
```

```
%% 7. Save Final Result to a Text File
fileID = fopen('plate_result.txt', 'w');
fprintf(fileID, '%s\n', final_output);
fclose(fileID);
```

خروجی کامل بخش اول

Image 1




```
Detected License Plate: DL5CH8855
>>
```

Image 2




```
Detected License Plate: DL2CAD0311
>>
```

Image 3




```
Detected License Plate: UP14CB7145
>>
```

قسمت دوم

پیش پردازش تصویر و حذف نویز های کوچک و بک گراند

```
%% 3. Image Cleanup
if size(picture, 3) == 3
  grayImage = rgb2gray(picture);
else
  grayImage = picture;
end
level = graythresh(grayImage);
binaryImage = imbinarize(grayImage, level);
binaryImage = ~binaryImage;
cleanImage = bwareaopen(binaryImage, 600);
largeObjects = bwareaopen(cleanImage, 6000);
cleanImage = cleanImage & ~largeObjects;
imshow(cleanImage);
```

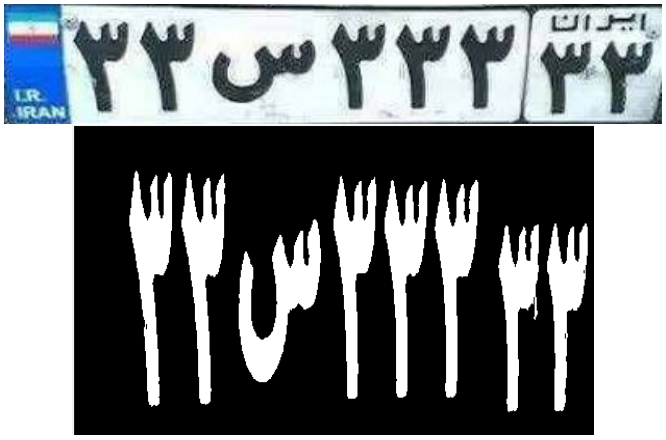
تصویر ورودی به منظور سادگی پردازش به حالت خاکستری تبدیل می‌شود. در اینجا از دستورات آماده متلب استفاده شده است؛ در صورتی که تصویر رنگی باشد (ماتریس سه‌بعدی)، تابع rgb2gray آن را به تک‌کاناله تبدیل می‌کند. سپس تابع graythresh بهترین سطح آستانه را به صورت خودکار محاسبه کرده و imbinarize تصویر را به حالت سیاه و سفید درآورده و در نهایت معکوس می‌گردد (متن سفید، پس‌زمینه مشکی).

حذف نویزهای کوچک و نقطه‌ها: با اجرای دستور bwareaopen(binaryImage, 600)، تمام اشیایی که کمتر از ۶۰۰ پیکسل مساحت دارند حذف می‌شوند. این کار علاوه بر حذف نویزها، باعث می‌شود نقطه‌های حروف فارسی نیز پاک شوند

حذف نویزهای بزرگ و کادرهای اضافی: برای حذف پس‌زمینه یا کادر پلاک، دستور bwareaopen(cleanImage, 6000) اشیایی که بیش از ۶۰۰۰ پیکسل دارند را شناسایی می‌کند. سپس با استفاده از عملگر & این عناصر از تصویر نهایی کم می شوند.

خروجی این بخش برای تصاویر نمونه:









بارگذاری دیتابیس

```
function TRAIN = load_mapset(folderName)
  files = [dir(fullfile(folderName, '*.bmp')); dir(fullfile(folderName, '*.png'))];
  len = length(files);
  TRAIN = cell(2, len);
  for i = 1:len
     img = imread(fullfile(folderName, files(i).name));
     if size(img, 3) == 3
        img = rgb2gray(img);
     end
     if ~islogical(img)
```

```
        img = imbinarize(img);
    end
    TRAIN{1, i} = img;
        [~, name, ~] = fileparts(files(i).name);
    splitName = split(name, ' ');
    TRAIN{2, i} = splitName{1};
  end
end
```

تابع load_mapset تمام فایل‌های .bmp و .png را می‌خواند. بررسی می‌کند که اگر تصویری رنگی است، خاکستری شده و اگر از قبل باینری نیست، باینری شود همچنین نام فایل‌ها با دستور split بررسی شده تا لیبل خالص کاراکتر به عنوان خروجی ثبت شود.

تشخیص کاراکترها

```
function plate_string = read_plate_chars(img, TRAIN)
  % 1. Segmentation
  [L, Ne] = bwlabel(img);
  plate_string = [];
  totalLetters = size(TRAIN, 2);
    % 2. Character Recognition
  for n = 1:Ne
    [r, c] = find(L == n);
    Y = img(min(r):max(r), min(c):max(c));
        [r_y, c_y] = find(Y == 1);
    if ~isempty(r_y) && ~isempty(c_y)
        Y = Y(min(r_y):max(r_y), min(c_y):max(c_y));
    end
    %imshow(Y);
        Y = imresize(Y, [42, 24]);
        ro = zeros(1, totalLetters);
```

```
    for k = 1:totalLetters
        ro(k) = corr2(double(TRAIN{1, k}), double(Y));
    end
        [MAXRO, pos] = max(ro);
    if MAXRO > 0.5
        out = TRAIN{2, pos};
        plate_string = [plate_string, out];
    end
  end
end
```

تصویر فیلتر شده به همراه دیتابیس، به تابع read_plate_chars ارسال می‌شود. در این تابع مراحل قطعه‌بندی با دستور bwlabel انجام می‌شود. به ازای هر شی، الگوریتم یک برش مماس انجام داده تا فقط پیکسل‌های حاوی اطلاعات باقی بمانند و سپس آن را به ابعاد استاندارد قالب‌ها (۴۲ در ۲۴) تغییر سایز می‌دهد. در نهایت، با محاسبه ضریب همبستگی دو بعدی (corr2) بین قطعه تصویر و دیتابیس، شبیه‌ترین کاراکتر انتخاب می‌شود. برای جلوگیری از ثبت نویزهای باقی‌مانده، تنها قطعاتی به رشته نهایی پلاک اضافه می‌شوند که شباهتشان با یکی از الگوها بیش از ۵۰ درصد (MAXRO > 0.5) باشد.

خروجی کامل بخش دوم



Detected License Plate: 27M13322



Detected License Plate: 33Sin33333



Detected License Plate: 16b93593



Detected License Plate: 14M12614

قسمت سوم

روش استخراج کادر پلاک از تصویر

برای این کار از روشی بر پایه تراکم لبه های عمودی و فیلترینگ هندسی استفاده شده است. این روش سه ویژگی اساسی برای پلاک در نظر گرفته که آن بخش را از سایر اجزا خودرو متمایز می کند:

۱. تراکم بالای لبه های عمودی: به دلیل وجود حروف و اعداد و خطوط متوالی مشکی روی پس زمینه سفید، پلاک خودرو دارای تغییرات شدید نوری در راستای افق یا لبه های عمودی است

۲. شکل مستطیلی پلاک

۳. نسبت ابعاد مشخص: طول پلاک های خودرو در ایران، به طور استاندارد تقریبا ۳ تا ۵ برابر عرض است

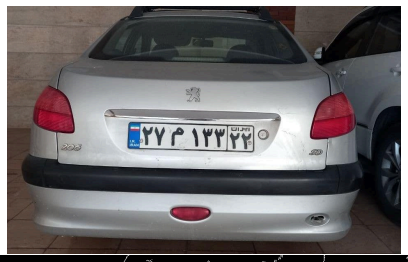
پیاده سازی مرحله به مرحله استخراج پلاک

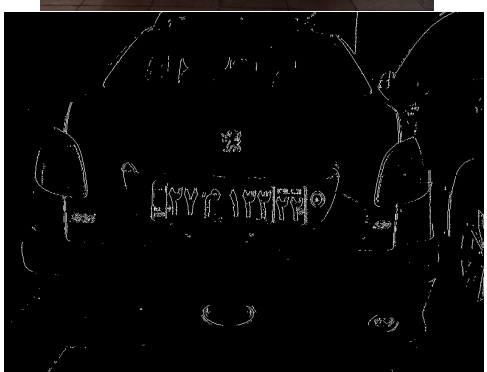
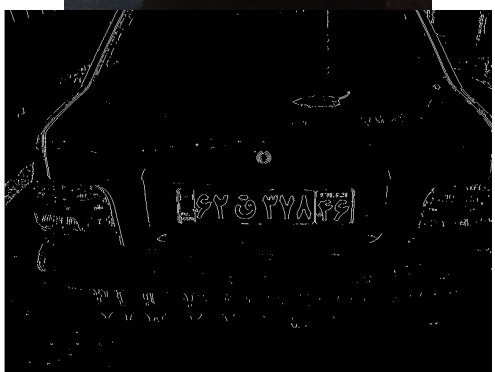
۱) استخراج لبه های عمودی

```
[~, threshold] = edge(grayImage, 'sobel', 'vertical');
edgeImage = edge(grayImage, 'sobel', threshold * 0.8, 'vertical');
```

تصویر به حالت خاکستری تبدیل می‌شود. در اینجا از فیلتر مشتق‌گیر سوبل با پارامتر 'vertical' استفاده شده است. این پارامتر باعث می‌شود فیلتر صرفا تغییرات نوری در راستای محور X را ثبت کند و خطوط افقی را نادیده بگیرد. ضریب 0.8 که در threshold ضرب شده است، حساسیت لبه‌یاب را ۲۰ درصد افزایش می‌دهد تا لبه‌های کمرنگ‌تر برخی اعداد نیز به خوبی شناسایی شوند. در خروجی این مرحله، ناحیه پلاک به صورت شبکه‌ای متراکم از خطوط سفید و عمودی ظاهر می‌شود.

خروجی این بخش برای تصاویر نمونه:

۲) عملیات مورفولوژیک

```
se = strel('rectangle', [12, 35]);
morphImage = imclose(edgeImage, se);
morphImage = imfill(morphImage, 'holes');
morphImage = bwareaopen(morphImage, 1000);
```

imclose: با استفاده از یک المان ساختاری (strel) به شکل مستطیل افقی با طول ۳۵ پیکسل، فواصل بین لبه‌های عمودی پلاک به هم متصل می‌شوند و یک کادر مستطیلی شکل می‌گیرد.
imfill: سوراخ‌های احتمالی داخل این بلوک سفید رنگ را پر می‌کند تا کاملا توپر شود.
bwareaopen: نواحی کوچکتر از ۱۰۰۰ پیکسل را پاک می‌کند تا فقط بلوک‌های بزرگ و اصلی باقی بمانند.

خروجی این بخش برای تصاویر نمونه:





۳) فیلترینگ هندسی

```
cc = bwconncomp(morphImage);
stats = regionprops(cc, 'BoundingBox', 'Area');
plate_img = [];
imshow(carImage);
for i = 1:cc.NumObjects
```

```
  bb = stats(i).BoundingBox; % [x, y, width, height]
  w = bb(3);
  h = bb(4);
  aspect_ratio = w / h;
   if aspect_ratio > 2.5 && aspect_ratio < 6.5 && w > 80 && w < 400 && h > 20
     rectangle('Position', bb, 'EdgeColor', 'g', 'LineWidth', 3);
     plate_img = imcrop(carImage, bb);
     break;
  end
end
```

دستور bwconncomp تمام بلوک‌های سفید باقی‌مانده را پیدا کرده و regionprops کادر محیطی آن‌ها را استخراج می‌کند که خروجی آن به فرمت [x, y, width, height] است.

سپس یک حلقه for روی تمام بلوک‌های پیدا شده اجرا می‌شود تا ویژگی‌های آن‌ها بررسی شود:

۱. نسبت عرض به ارتفاع کادر باید مطابق با استاندارد پلاک باشد.

۲. عرض و ارتفاع کادر باید از یک حداقل منطقی بزرگتر باشد تا نویزهای ریز به عنوان پلاک انتخاب نشوند.

۳. عرض باید از یک مقدار حداکثر منطقی کوچک تر باشد

بعد از اینکه اولین بلوک با این ویژگی‌ها یافت شد، کادر سبز روی آن رسم شده، با دستور imcrop از تصویر اصلی بریده می‌شود و حلقه متوقف می شود

خروجی این بخش برای تصاویر نمونه:







اسکریپت کامل p3.m

```
clc;
clear;
close all;

%% 1. Load and Resize Image
[file, path] = uigetfile({'*.jpg;*.jpeg;*.bmp;*.png'}, 'Choose a car image');
if isequal(file, 0)
    return;
end
carImage = imread(fullfile(path, file));
carImage = imresize(carImage, [600, 800]);

%% 2. Extract plate
plateImage = extract_plate(carImage);
%imshow(plateImage);

%% 3. Image cleanup
plateImage = imresize(plateImage, [300, 500]);
%figure; imshow(plateImage);
cleanImage = clean_image(plateImage, 900, 6000);
figure; imshow(cleanImage);

%% 4. Load data base
TRAIN = load_mapset('Map set Farsi');

%% 5. Read Plate Characters
final_output = read_plate_chars(cleanImage, TRAIN);

%% 6. Display and Save Result
disp(['Detected License Plate: ', final_output]);

fileID = fopen('plate_result_p2.txt', 'w', 'n', 'UTF-8');
fprintf(fileID, '%s\n', final_output);
fclose(fileID);
```

خروجی کامل بخش سوم



Detected License Plate: 59Y84410



Detected License Plate: 27M13322



Detected License Plate: 62Gh37846

قسمت چهارم

در این قسمت از پروژه، هدف پردازش یک فایل ویدیویی پیوسته به جای تصاویر ثابت است. برای این منظور، الگوریتم باید بتواند در هر فریم لرزش‌های دوربین را خنثی کند، خودروهای در حال حرکت را از پس‌زمینه جدا کرده و رهگیری کند، سرعت حرکت آن‌ها را تخمین بزند و در نهایت با استخراج پلاک، کاراکترهای آن را با استفاده از دیتابیس (Template Matching) شناسایی کند.

تثبیت فریم‌های ویدیو

```
function stabilizedFrame = stabilize_frame(currentFrame, referenceFrame)
  currGray = rgb2gray(currentFrame);
  refGray = rgb2gray(referenceFrame);
    ptsRef = detectSURFFeatures(refGray);
  ptsCurr = detectSURFFeatures(currGray);
    [featuresRef, validPtsRef] = extractFeatures(refGray, ptsRef);
  [featuresCurr, validPtsCurr] = extractFeatures(currGray, ptsCurr);
    indexPairs = matchFeatures(featuresRef, featuresCurr);
    if length(indexPairs) > 10
      matchedRef = validPtsRef(indexPairs(:, 1));
      matchedCurr = validPtsCurr(indexPairs(:, 2));
      tform = estimateGeometricTransform2D(matchedCurr, matchedRef, 'affine');
      stabilizedFrame = imwarp(currentFrame, tform, 'OutputView',
imref2d(size(currentFrame)));
  else
      stabilizedFrame = currentFrame;
  end
end
```

توضیح تابع stabilize_frame:

ابتدا فریم فعلی و فریم مرجع (فریم قبلی) به حالت خاکستری تبدیل می‌شوند. سپس با استفاده از دستور detectSURFFeatures نقاط کلیدی هر دو تصویر استخراج شده و با دستور matchFeatures با یکدیگر تطبیق داده می‌شوند. اگر تعداد نقاط مشترک پیدا شده کافی باشد (بیشتر از ۱۰ نقطه)،

ماتریس انتقال هندسی (Affine Transform) با دستور estimateGeometricTransform2D محاسبه شده و با اعمال imwarp، فریم فعلی دقیقا روی فریم قبلی تراز می‌شود.

تشخیص حرکت و استخراج خودروها

```
function cleanMask = clean_foreground(currGray, prevGray, diffThresh, closeSize, minArea)
  % Initialize foreground extraction based on motion and edges
    % 1. Motion Detection via Frame Differencing
  diffImg = abs(double(currGray) - double(prevGray));
  motionMask = diffImg > diffThresh;
    % 2. Spatial Edge Detection using Sobel operator
  edgeMask = edge(currGray, 'sobel');
    % Dilate edges to bridge small gaps and form continuous vehicle structures
  dilatedEdges = imdilate(edgeMask, strel('disk', 5));
    % 3. Intersection of Motion and Structural Edges
  rawMask = motionMask & dilatedEdges;
    % 4. Morphological Cleaning and Closing
  rawMask = imopen(rawMask, strel('disk', 2));
  rawMask = imclose(rawMask, strel('rectangle', closeSize));
  rawMask = imfill(rawMask, 'holes');
    % 5. Remove Small Noise Regions based on Area Threshold
  cleanMask = bwareaopen(rawMask, minArea);
    % 6. One-Sided Downward Expansion (To guarantee the license plate is included)
  padBottom = 40; % Number of pixels to expand downwards
  padSides = 15;  % Number of pixels to expand horizontally
    halfPad = round(padBottom / 2);
    % Create symmetric structuring element for initial expansion
  se = strel('rectangle', [halfPad * 2 + 1, padSides]);
  dilatedMask = imdilate(cleanMask, se);
```

```
    % Shift the expanded mask downwards to restore the top edge and stretch the
bottom edge
  shiftedMask = imtranslate(dilatedMask, [0, halfPad]);
    % Combine with the original clean mask to finalize the region
  cleanMask = cleanMask | shiftedMask;
 end
```

توضیح تابع clean_foreground:

در این بخش ابتدا با محاسبه قدر مطلق تفاضل فریم فعلی و قبلی و اعمال یک مقدار آستانه ، ماسک حرکتی استخراج می‌شود. سپس برای تشکیل ساختار یکپارچه خودرو، این ماسک با لبه‌های استخراج شده توسط فیلتر سوبل ترکیب می‌گردد. عملیات مورفولوژیک (imclose، imopen و imfill) سوراخ‌های درون شیء را پر کرده و بلوک‌های پیوسته ایجاد می‌کنند.

گسترش یک‌طرفه کادر: برای تضمین اینکه پلاک خودرو (که معمولاً در پایین‌ترین نقطه سپر قرار دارد) به طور کامل داخل کادر شناسایی شده قرار بگیرد، با استفاده از توابع imdilate و imtranslate ماسک خودرو به صورت هوشمند و نامتقارن (فقط به سمت پایین و کمی اطراف) گسترش می‌یابد تا مختصات دقیق برش در مرحله بعد تضمین شود.

محاسبه سرعت خودروها

```
function speedStr = calculate_speed(currCentroid, prevCentroids, dt, maxDist)
  speedStr = '0 px/s';
   if ~isempty(prevCentroids)
     distances = sqrt((prevCentroids(:, 1) - currCentroid(1)).^2 + (prevCentroids(:, 2) -
currCentroid(2)).^2);
     [minDist, ~] = min(distances);
          if minDist < maxDist
        speed = minDist / dt;
        speedStr = sprintf('%.1f px/s', speed);
     end
  end
end
```

توضیح تابع calculate_speed:

در این تابع، فاصله اقلیدسی بین مرکز جرم فعلی خودرو و تمامی مراکز جرم ثبت شده از خودروها در فریم قبلی محاسبه می‌شود. سپس با پیدا کردن کمترین فاصله ، بررسی می‌شود که آیا این فاصله از یک مقدار آستانه مشخص کمتر است یا خیر. این آستانه، که در این اسکریپت ۱۰۰ پیکسل در نظر گرفته شده است، نقش یک فیلتر منطقی را ایفا می‌کند تا از خطاهای رهگیری جلوگیری شود. به عنوان مثال، اگر خودروی جدیدی وارد فریم شود که در فریم قبلی حضور نداشته، و یا اگر خودرویی به طور موقت به دلیل انسداد ردیابی نشود، این فیلتر مانع از آن می‌شود که الگوریتم به اشتباه کمترین فاصله را نسبت به یک خودروی کاملاً متفاوت در فریم قبل محاسبه کرده و سرعت غیرواقعی نمایش دهد. در صورت عبور از این شرط، فاصله به دست آمده بر اختلاف زمان بین دو فریم تقسیم شده و سرعت بر حسب پیکسل بر ثانیه محاسبه می‌شود. در صورتی که مرکز جرمی در فریم قبل یافت نشود یا فاصله بیش از حد مجاز باشد، سرعت اولیه '0 px/s' بازگردانده می‌شود.

استخراج کادر پلاک با پنجره لغزان و خواندن کاراکترها

توضیح عملکرد و منطق الگوریتم در تابع extract_plate

۱. مقداردهی اولیه، اعتبارسنجی و تغییر ابعاد

ابتدا بررسی می‌شود که تصویر ورودی خالی یا بیش از حد کوچک نباشد. سپس برای اینکه الگوریتم به ابعاد مختلف تصاویر وابستگی نداشته باشد و فیلترها همیشه عملکرد یکسانی داشته باشند، عرض تصویر با حفظ تناسب ابعاد دقیقا به ۵۰۰ پیکسل تغییر می‌یابد. مقیاس تغییرات برای استفاده در مرحله برش نهایی ذخیره می‌شود.

۲. پردازش رنگ و استخراج نقشه انرژی متن

در این مرحله دو نقشه مهم از تصویر استخراج می‌شود:

- نقشه رنگ آبی: با کم کردن بیشینه مقادیر کانال‌های قرمز و سبز از کانال آبی، یک نقشه خالص از رنگ آبی استخراج می‌شود که هدف آن شناسایی نوار آبی‌رنگِ سمت چپ پلاک‌های ایرانی است.
- نقشه انرژی متن : با استفاده از کانولوشن یک‌بعدی ، تغییرات روشنایی در راستای افقی و عمودی محاسبه می‌شود. سپس با فرمول horizDiff - 0.5 * vertDiff، خطوط عمودیِ متراکم (مشخصه اصلی حروف و اعداد پلاک) تقویت شده و شبکه‌های متقاطع (مانند توری جلو پنجره که هم خطوط افقی و هم عمودی دارد) به شدت تضعیف و حذف می‌شوند.

۳. ساخت تصاویر انتگرالی

برای اینکه سرعت جستجو در تصویر به حداکثر برسد، از دستور integralImage روی نقشه انرژی، تصویر خاکستری و نقشه رنگ آبی استفاده می‌شود. این تکنیک ریاضی اجازه می‌دهد تا مجموع مقادیر پیکسل‌های داخل هر مستطیلی با هر ابعادی، تنها با چهار عمل جمع و تفریق ساده و در زمان $O(1)$ محاسبه شود.

۴. جستجوی پنجره لغزان و امتیازدهی

الگوریتم شبکه‌ای از مستطیل‌ها را روی نیمه پایینی تصویر می‌لغزاند. برای پوشش دادن فواصل مختلف دوربین، کادرها در ۳ عرض مختلف و ۲ نسبت ابعاد متفاوت تولید می‌شوند. به ازای هر کادر، یک امتیاز ترکیبی بر اساس ویژگی‌های زیر محاسبه می‌شود:

- انرژی متن (sumEnergy): میانگین تراکم لبه‌های متن در کل کادر.
- روشنایی هسته مرکزی (meanBright): میانگین روشنایی در مرکز کادر که باعث می‌شود بلوک‌های سیاه مانند لاستیک‌ها و جلوپنجره‌های تاریک امتیاز نزدیک به صفر بگیرند.
- حضور نوار آبی (meanBlue): بررسی وجود رنگ آبی خالص در ۲۲ درصد ابتدایی (سمت چپ) کادر.
- سوگیری فضایی (verticalBias و centerBias): کادرهایی که در مرکز افقی و پایین تصویر قرار دارند، امتیاز بالاتری دریافت می‌کنند. این ویژگی‌ها در یکدیگر ضرب می‌شوند و کادری که بالاترین امتیاز را کسب کند به عنوان پلاک انتخاب می‌شود.

۵. تبدیل مختصات، حاشیه‌سازی نامتقارن و برش

پس از یافتن بهترین کادر، برای جلوگیری از بریده شدن لبه‌های حروف یا اعداد کناری پلاک، یک حاشیه به کادر اضافه می‌شود. این حاشیه به صورت نامتقارن طراحی شده است؛ به طوری که حاشیه سمت راست بزرگ‌تر از حاشیه سمت چپ در نظر گرفته می‌شود تا انتهای پلاک به صورت کامل در کادر قرار گیرد. در نهایت، مختصات کادر با استفاده از متغیر scale به ابعاد تصویر اصلی بازگردانده شده و پلاک برش می‌خورد.

تابع کمکی rectSum

این تابع وظیفه محاسبه سریع مجموع مقادیر داخل یک مستطیل را از روی تصاویر انتگرالی بر عهده دارد. با استفاده از دستور sub2ind، مختصات ماتریسی چهار گوشه مستطیل به اندیس‌های خطی تبدیل می‌شوند تا متلب بتواند در کسری از ثانیه میلیون‌ها کادر را بدون استفاده از حلقه‌های کُند پردازش کند.

نمایش و ذخیره نتیجه نهایی ویدیو

```
    % Draw Annotations
    displayFrame = insertShape(displayFrame, 'Rectangle', carBBox, 'Color', 'green', 'LineWidth', 3);
    textPosSpeed = [carBBox(1) + carBBox(3) + 10, carBBox(2)];
    displayFrame = insertText(displayFrame, textPosSpeed, speedStr, 'FontSize', 18, 'BoxColor', 'yellow');
    textPosPlate = [carBBox(1), max(carBBox(2) - 40, 1)];
    displayFrame = insertText(displayFrame, textPosPlate, plateText, 'FontSize', 22, 'BoxColor', 'white');
```

با استفاده از دستور insertShape کادر محیطی سبز رنگ به دور خودرو رسم می‌شود. همچنین دستور insertText برای چاپ مقدار سرعت در سمت راست کادر و رشته پلاک خوانده شده در بالای کادر استفاده می‌گردد. در انتها، فریم‌های پردازش شده در دو فایل ویدیویی output_clean.avi (فقط تصویر اصلی و کادرها) و output_final.avi (تصویر اصلی در کنار ماسک باینری) از طریق VideoWriter ذخیره می‌شوند.

اسکریپت کامل p4.m

```
% 1. Initialization & Parameters
TRAIN = load_mapset('Map set Farsi');
videoFile = 'SampleVideos/video3.avi';
vidReader = VideoReader(videoFile);
fps = vidReader.FrameRate;
dt = 1 / fps;

% Define output video files
outputCombinedFile = 'Result/output_final.avi';          % Combined video (original frame + mask)
outputCleanFile = 'Result/output_clean.avi';             % Clean final video (vehicles and plates only)

vidWriterCombined = VideoWriter(outputCombinedFile);
vidWriterCombined.FrameRate = fps;
open(vidWriterCombined);

vidWriterClean = VideoWriter(outputCleanFile);
vidWriterClean.FrameRate = fps;
open(vidWriterClean);

% VideoPlayer removed for direct background video writing without frame-by-frame display

prevFrame = readFrame(vidReader);
prevGray = rgb2gray(prevFrame);
prevCentroids = [];
diffThresh = 70;
closeSize = [80, 15];
minArea = 5000;
vidReader.CurrentTime = 0.3;

% 2. Main Processing Loop
while hasFrame(vidReader)
    currFrame = readFrame(vidReader);

    % Stabilization & Foreground Extraction
    stabilizedCurr = stabilize_frame(currFrame, prevFrame);
    currGray = rgb2gray(stabilizedCurr);
    cleanMask = clean_foreground(currGray, prevGray, diffThresh, closeSize, minArea);

    cars = regionprops(cleanMask, 'BoundingBox', 'Centroid', 'Area');
    displayFrame = currFrame;
    currCentroids = zeros(length(cars), 2);

    % 3. Vehicle Tracking & ALPR
    for i = 1:length(cars)
        carBBox = cars(i).BoundingBox;
        currCentroid = cars(i).Centroid;
        currCentroids(i, :) = currCentroid;

        % Speed Estimation
        speedStr = calculate_speed(currCentroid, prevCentroids, dt, 100);

        % Plate Extraction & OCR
        carImg = imcrop(currFrame, carBBox);
        % imshow(carImg); % Commented out to prevent blocking batch video processing

        plateImg = extract_plate(carImg);
        plateText = "";

        if ~isempty(plateImg)
            cleanedPlate = clean_image(plateImg, 10, 110);
            % imshow(cleanedPlate); % Commented out
            plateText = read_plate_chars(cleanedPlate, TRAIN);
        end

        if isempty(plateText)
            plateText = 'Unknown';
        end

        % Draw Annotations
        displayFrame = insertShape(displayFrame, 'Rectangle', carBBox, 'Color', 'green', 'LineWidth', 3);
        textPosSpeed = [carBBox(1) + carBBox(3) + 10, carBBox(2)];
        displayFrame = insertText(displayFrame, textPosSpeed, speedStr, 'FontSize', 18, 'BoxColor', 'yellow');
        textPosPlate = [carBBox(1), max(carBBox(2) - 40, 1)];
        displayFrame = insertText(displayFrame, textPosPlate, plateText, 'FontSize', 22, 'BoxColor', 'white');
    end
```

```
    % 4. Update Variables & Write to Both Video Files
    prevCentroids = currCentroids;
    maskRGB = repmat(uint8(cleanMask) * 255, [1, 1, 3]);
    combinedFrame = [displayFrame, maskRGB];

    % Write frames to both output video writers
    writeVideo(vidWriterCombined, combinedFrame);
    writeVideo(vidWriterClean, displayFrame);

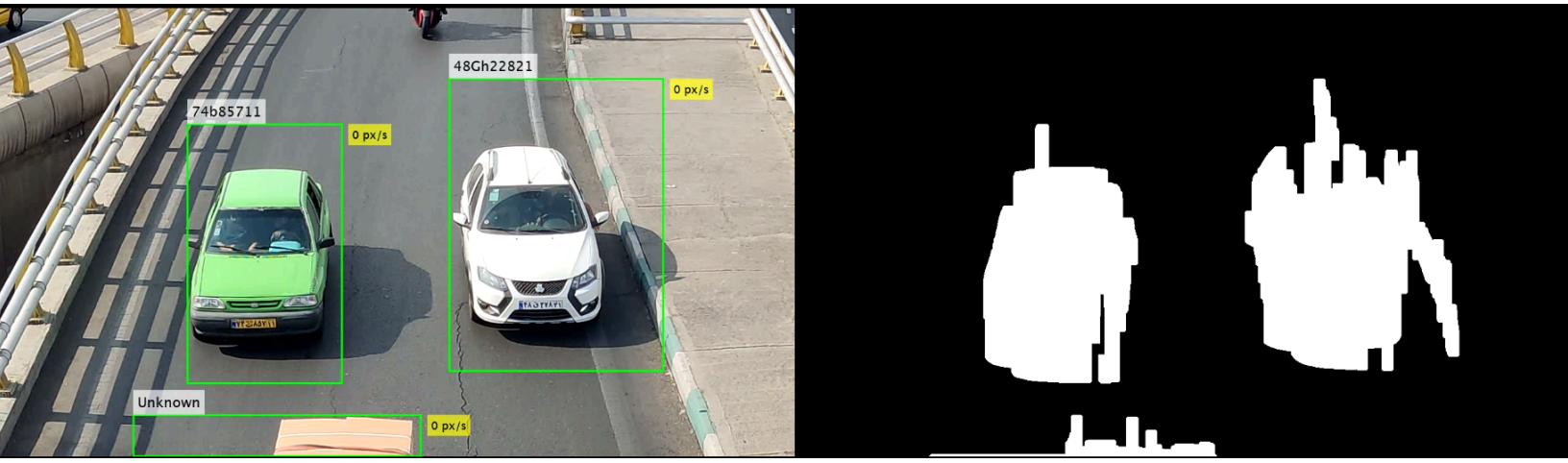
    prevFrame = stabilizedCurr;
    prevGray = currGray;

    %step(vidPlayer, combinedFrame);
end

% Close video writers
close(vidWriterCombined);
close(vidWriterClean);

disp('Processing completed. Check output_final.avi and output_clean.avi.');
```

خروجی بخش چهارم



قسمت پنجم

توضیح استراتژی حل مسئله

برای این مسئله از الگوریتم template matching بر پایه ضریب همبستگی نرمالایز شده استفاده شده است. علت استفاده از ضریب نرمالایز شده این است که بخش هایی که دامنه پیکسلی بالاتری دارند (بخش های روشن تر board) اما با الگو همخوانی ندارند به اشتباه ضریب همبستگی بیشتری نداشته باشند و صرفا شکل قطعه مقایسه شود.

توضیح تابع ICrecognition

```
function ICrecognition(ICImage, PCBImage)
    figure;
    imshow(PCBImage);
    hold on;

    % Convert to grayscale
    if size(ICImage, 3) == 3
        ic_gray = rgb2gray(ICImage);
    else
        ic_gray = ICImage;
    end

    if size(PCBImage, 3) == 3
        pcb_gray = rgb2gray(PCBImage);
    else
        pcb_gray = PCBImage;
    end

    threshold = 0.7;

    % Without rotation
    [ic_h, ic_w] = size(ic_gray);
    corr = normxcorr2(ic_gray, pcb_gray);

    [y_peaks, x_peaks] = find(corr >= threshold);

    for i = 1:length(x_peaks)
        x = x_peaks(i) - ic_w;
        y = y_peaks(i) - ic_h;
        rectangle('Position', [x, y, ic_w, ic_h], 'EdgeColor', 'b', 'LineWidth', 2);
    end

    % With rotation (180 degrees)
    ICImageRotated = imrotate(ic_gray, 180);
    corr_180 = normxcorr2(ICImageRotated, pcb_gray);

    [y_peaks_180, x_peaks_180] = find(corr_180 >= threshold);

    for i = 1:length(x_peaks_180)
        x = x_peaks_180(i) - ic_w;
        y = y_peaks_180(i) - ic_h;
        rectangle('Position', [x, y, ic_w, ic_h], 'EdgeColor', 'b', 'LineWidth', 2);
    end

    hold off;
end
```

پیش‌پردازش تصاویر: از آنجا که تابع normxcorr2 بر روی ماتریس‌های دو بعدی عمل می‌کند، در ابتدای تابع بررسی می‌شود که اگر تصاویر ورودی (IC و PCB) رنگی هستند، به حالت خاکستری تبدیل شوند تا محاسبات بدون خطا انجام شود.

تشخیص قطعات در حالت بدون دوران: ابتدا تصویر IC بدون هیچ تغییری روی کل تصویر PCB لغزانده شده و ماتریس همبستگی محاسبه می‌گردد. خروجی این ماتریس اعدادی بین ۱- تا ۱+ است. با استفاده از دستور find(corr >= threshold)، پیک‌هایی که تطابق بالای ۷۰ درصد با الگو دارند به عنوان مکان‌های قطعه صفر درجه استخراج می‌شوند.

تشخیص قطعات در حالت دوران یافته: تصویر الگو با دستور imrotate(ic_gray, 180) به اندازه ۱۸۰ درجه دوران می یابد. سپس correlation دوباره محاسبه می شود. مانند بخش قبل پیک هایی با شباهت بالای ۷۰ درصد استخراج می شوند.

محاسبه مختصات و رسم کادر: خروجی تابع normxcorr2 مختصات گوشه پایین سمت راست قالب پیدا شده است. برای استفاده از دستور rectangle جهت رسم مستطیل، باید مختصات گوشه بالا سمت چپ را داشته باشیم. بنابراین، طول و عرض تصویر الگو (ic_w و ic_h) از مختصات به دست آمده کم می‌شود. در نهایت قطعات پیدا شده در هر دو حالت با مستطیل‌های آبی روی تصویر رنگی مشخص می شوند.

اسکریپت p5.m

```
% Load images
ICImage = imread('IC.png');
PCBImage = imread('PCB.jpg');

% ic recognition
ICrecognition(ICImage, PCBImage);
```

خروجی بخش پنجم

